

Entelechy Continuum Engine

Corrected kernels, catalog, and build cut. Version 0.1.0.

This handbook compiles the design documents in docs/. Executable truth is ece/*.py and tests/. The original catalog remains lineage reference; formulas here supersede it.

Entelechy Continuum Engine — Overview

ECE is a hybrid Eulerian–Lagrangian artificial-life runtime. It does not collapse every rule family into one equation. It runs them as plugins on a shared state:

- Eulerian fields (Lenia, SmoothLife, Field Life, MaCE, RD)
- Lagrangian particles (Mohr Particle Life, Clusters, PPS, Boids)
- Particle-in-Cell coupling (deposit / sample) for hybrids

This repository currently ships the **corrected reference kernels** and the design documents. GPU host, renderer, and co-local AI are specified here and deferred until the kernels stay green.

Two axes that must not be mixed

Lagrangian vs Eulerian — points moving through space vs density on a grid.

Conserving vs open-growth — mass invariant (MaCE, Flow-Lenia, Field Life) vs source/sink (classic Lenia, GoL, Mohr).

Asymmetric species matrices violate momentum conservation on purpose. That is how chase rings and organelles appear.

Source of truth

Layer	File
Spec deltas vs the original catalog	CORRECTIONS.md
Executable math	ece/*.py
GPU fragments	shaders/*.wgsł
Invariants	tests/
Family catalog	docs/02-rules-catalog.md
Build cut	docs/09-roadmap.md

Default first run (M1)

2D Mohr Particle Life, torus world, 6 species, live matrix, spatial hash, exponential friction. No 3D, no VLM, no FFT.

Lineage

Tom **Mohr** of particle-life.com, not “Tom Moore.” Clusters (Jeffrey Ventrella) came first. Mohr simplified the per-pair piecewise fields into one global tent plus one matrix. Field Life moves that matrix onto density fields and adds MaCE so mass does not evaporate. Lenia is a different line: Conway’s Life made continuous.

GoL (B3/S23)

- ■ Life-like CA (B/S notation)
- ■ Larger-than-Life / Bugs
- ■ SmoothLife (disk + annulus + birth/survive intervals)
- ■ ■ SmootherLife (Gaussian kernels)
- ■ Lenia (radial kernel + growth bell)
 - ■ multi-channel / multi-kernel
 - ■ Glaberish (genesis + persistence)
 - ■ Flow-Lenia (mass-conserving flow)
 - ■ MaCE-Lenia
 - ■ Particle Lenia (energy on dots)

Clusters (Ventrella): per-pair (r1,r2,f1,f2) + asymmetric horizons

- ■ Particle Life (Mohr): one tent + one matrix
- ■ ■ hunar4321 / najarro.science / par-particle-life
- ■ ■ Field Life = matrix + kernel + MaCE
- ■ Clusters X / Atomic Clusters (ciphrd)

PPS (Schmickl): $\Delta\phi = \alpha + \beta N \text{ sign}(R - L)$

Boids (Reynolds): separate / align / cohere

NCA: learned local net

Reaction-diffusion: Gray-Scott, FitzHugh-Nagumo

What each ancestor contributes

Ancestor	Keep
Clusters	Asymmetric matrix, optional per-pair radii later
Mohr	Global tent, one r_max , one beta , spatial bins
Field Life	Convolution + matrix + conservative transport
Lenia	Kernel + unimodal growth, optional multi-channel
MaCE	Softmax-9 / softmax-27 mass routing
PPS / Boids	Heading agents; same bins, different force pass

Particle Lenia is the Lagrangian reading of Lenia’s energy, not a Mohr variant. Do not implement it until deposit/sample exists.

Rules catalog

State, interaction, and conservation. Implementation parameters live in `configs/` and `schemas/sim.schema.json`.

Family table

Family	State	Interaction	Conserves mass?
Life-like CA	discrete cells	neighbor count	no
SmoothLife / Lenia	continuous field	convolution + growth	no (unless Flow/MaCE)
Clusters / Particle Life	colored points	pairwise force matrix	no
Field Life	colored density fields	kernel + matrix + transport	yes, per color
Particle Lenia	points	energy gradient	no, unless added
PPS / Boids	heading agents	local geometry	no

Side-by-side

System	Substrate	State	Topology	Primary parameters	Conserves	Cost per step
GoL / Life-like	discrete grid	{0,1}	Moore 8	B / S counts	no	O(N)
SmoothLife	field	[0,1]	disk + annulus	radii, birth/survive intervals, α	no	O(N log N) FFT
Lenia	field	[0,1]	radial kernel	shells β , (μ, σ) , dt	no	O(N log N)
Flow-Lenia / MaCE-Lenia	field	[0,1]	kernel + flux	Lenia + transport β	yes	O(N log N)+O(N)
Clusters	particles	x,v,c	pairwise, directed	per-pair (r1,r2,f1,f2), γ , delay	no	O(M k) bins
Particle Life (Mohr)	particles	x,v,c	pairwise $\leq r_{\max}$	matrix A, β , λ	no	O(M k) hash
Field Life	density grid	$\rho_c \geq 0$	conv + matrix + MaCE	K, M, λ_{crowd} , β_{tr}	yes	O(K N log N)
Particle Lenia	particles	x	KDE + energy	kernel, growth, c_rep	no	O(M k)
PPS	agents	x, ϕ	left/right count	r, α , β , v	no	O(M k)
Boids / Swarm Chemistry	agents	x, v	neighborhood means	w_sep, w_align, w_coh	no	O(M k)

Update rules (compressed)

Conway / Life-like

Birth on count set B, survive on S. Classic B3/S23.

SmoothLife

Inner disk fill m , outer annulus fill n , logistic mix of birth interval $[b1, b2]$ and survival $[d1, d2]$. Famous glider: birth $[0.278, 0.365]$, survive $[0.267, 0.445]$, $\alpha_n \approx 0.028$, $\alpha_m \approx 0.147$.

Lenia

$U = K * A$, $G(u) = 2 \exp(-(u-\mu)^2 / 2\sigma^2) - 1$, $A \leftarrow \text{clip}(A + dt \ G(U), \ 0, \ 1)$. Orbium: ring near $r=0.5$, $\mu \approx 0.15$, $\sigma \approx 0.015$.

Mohr Particle Life (corrected)

See docs/03-mohr.md. Tent + matrix. No extra r_{max} on the unit vector.

Field Life / MaCE (corrected)

See docs/04-mace.md. Two-pass softmax transport on a torus.

PPS

$\Delta\phi = \alpha + \beta \ N \ \text{sign}(R-L)$, constant speed. Default life-like: $r=5$, $\alpha=180^\circ$, $\beta=17^\circ$, $v=0.67$.

Boids

Separation, alignment, cohesion. Swarm Chemistry assigns different weight triples per species — same “matrix of types” idea, geometric steering instead of a radial tent.

DAG passes per rule

Rule	Passes
Mohr / Clusters	Forces, Integrate
Field Life	Convolve, TransportDenom, TransportGather
Lenia / SmoothLife	Convolve, Growth
Flow-Lenia / MaCE-Lenia	Convolve, Growth, Transport*
Particle Lenia	Deposit, Convolve, Sample, Integrate
PPS / Boids	Steer, Integrate
Hybrid PIC	Deposit + any of the above + Sample

Mohr Particle Life — corrected force

Particles: position x , velocity v , species $c \in \{0..K-1\}$. One global r_max , one global wall $\beta \in (0,1)$, one matrix $A[K,K] \in [-1,1]$, optional scalar gain (default 1).

Tent

Normalize $r = ||x_j - x_i||_wrap / r_max$.

$$F(r, a) = \begin{cases} r/\beta - 1 & \text{if } r < \beta \\ a * (1 - |2r - 1 - \beta| / (1-\beta)) & \text{if } \beta \leq r \leq 1 \\ 0 & \text{if } r > 1 \end{cases}$$

- $r < \beta$: everyone repels everyone. F does not depend on a .
- Mid-range peak is at $r = (1+\beta)/2$ and equals a .
- Horizon is r_max . Wrapped shortest vector on the torus.

Acceleration (the correction)

$$a_i = \text{gain} * \sum_{j \neq i} F(r_{ij}, A[c_i, c_j]) * \hat{(x_j - x_i)}$$

Do **not** multiply by r_max . That factor made peak force depend on world units and broke “same matrix, different r_max ” comparisons.

`ece.mohr.mohr_accelerations` is the $O(N^2)$ oracle used by tests. Production uses the spatial hash in `docs/05-bins-integrate.md`.

Integration

$$\begin{aligned} v &\leftarrow \exp(-\lambda \, dt) \, v + a \, dt \\ x &\leftarrow \text{wrap}(x + v \, dt) \end{aligned}$$

$\lambda = 0$ is undamped. To recover Mohr’s $\gamma^{**}(60 \, dt)$ feel:

$$\lambda = -\ln(\gamma) * 60$$

See `ece.integrate`.

Asymmetry

A is not required to be symmetric. $A[i, j] \neq A[j, i]$ is the mechanism for orbits and chase rings. Total momentum is not a conserved quantity and must not be treated as a test invariant.

Clusters extension (later)

Replace the single tent with per-pair $(r1, r2, f1, f2)$ and optional view-radius / delay. Keep $f1$ non-attractive so particles cannot collapse through the wall.

Tests that lock this

- wall independent of matrix entry and strictly negative
- tent peak equals a at midpoint
- $F = 0$ beyond horizon

- scale world and `r_max` together → identical accelerations
- asymmetric chase pair produces net momentum

MaCE / Field Life — two-pass torus transport

Field Life keeps Mohr’s matrix, drops the dots, and moves mass on a grid so numerical friction cannot evaporate creatures.

Sense (affinity)

Each color c is a field $p_c \geq 0$.

$$A_c = \text{strength} * \sum_d M[c,d] (K * p_d) - \lambda_{\text{crowd}} p_c$$

Default K is a signed ring (negative near 0, positive on a shell) so hollow shells beat solid blobs. Convolution is periodic.

Move (MaCE)

Pass A — denominators

$$Z[x] = \sum_{\{y \in N(x)\}} \exp(\beta A[y])$$

Pass B — gather

$$p'[x] = \sum_{\{y \in N(x)\}} p[y] * \exp(\beta A[x]) / Z[y]$$

$N(x)$ is the 9-neighborhood (2D) or 27-neighborhood (3D), **including self**, wrapped on the torus.

What was wrong in the catalog shader

Recomputing neighbor softmax denominators inside the gather without a stored Z .

continue on out-of-bounds instead of wrap — mass leaks at the box face.

No unit test that $\sum p'_c = \sum p_c$.

Invariants

- Per-channel mass is bit-stable within float rounding.
- Momentum is not conserved.
- A uniform field is a fixed point when the kernel mean is 0 and crowding is applied evenly.
- $Z[x] > 0$ always (exp is positive; neighborhood is non-empty).

Implementation

Python: `ece.mace.mace_step`. WGSL: `shaders/mace_2pass.wgsl` (`mace_denom`, `mace_gather`).

Multi-channel: run A and B per channel. Z is computed from that channel’s affinity, not shared across colors, unless you deliberately couple transport.

3D

Same formulas, 27 neighbors, `np.roll` on three spatial axes. Do not start 3D until 2D mass tests are boring.

Spatial hash and integration

Neighbor search must be $O(Mk)$, not $O(M^2)$. Uniform bins are enough for a torus with a single `r_max`.

Cell size

`cell_size >= r_max` lets a 3×3 (2D) or $3 \times 3 \times 3$ (3D) walk cover the horizon. If memory for bins explodes, coarsen to `cell = 2 r_max` and walk 5×5 . Document the choice; do not mix them at runtime.

Four arrays, not two

Buffer	Role
<code>counts</code>	frozen particle-per-bin snapshot
<code>offsets</code>	exclusive prefix sum of <code>counts</code>
<code>ranges</code>	<code>(start, end) = (offsets[b], offsets[b]+counts[b])</code>
<code>cursor</code>	scatter write head, discarded after the pass

`sorted_indices[start:end]` are the particles in that bin.

Never exclusive-scan a counter you later `atomicAdd` as a cursor. The catalog mixed those roles.

Algorithm

Clear `counts`.

Count: `counts[bin(x_i)] += 1`.

Snapshot `counts`. Exclusive scan $\rightarrow$ `offsets`.

Pack `ranges`.

Scatter into `sorted_indices` using a copy of `offsets` as cursor.

Force pass walks neighbor bins via `ranges`.

Integrate.

`ece.bins.snapshot_and_scatter` is the CPU oracle.

Integration

Symplectic Euler + wrap:

```
decay = exp(-λ dt)
v ← decay v + a dt
x ← x + v dt
x ← x - floor(x / world) * world
```

World is a unit torus unless a config says otherwise. Positions must stay in $[0, \text{world})$.

Determinism note

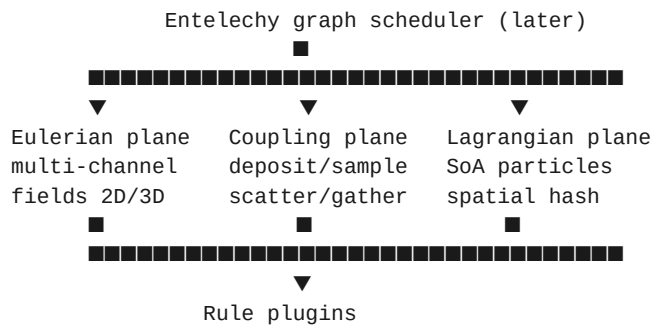
Float GPU atomics and vendor exp make wgpu preview non-replayable. If replay logs matter, freeze:

- particle order after scatter (already sorted by bin then arrival)
- neighborhood walk order (fixed nested dx,dy)
- a fixed-point / LUT path on CUDA later

Hash state = frame counter + positions + types + matrix. Do not hash accel, Z, or ranges.

Architecture

Planes



Shared state (logical)

```

time, dt
euler: [C, X, Y, (Z)] f32 ping-pong or absent
lagrange: pos, vel, type, mass, heading or absent
matrix: K×K f32, optional per-pair radii
kernels: radial samples + optional FFT plan
hash: counts, offsets, ranges, sorted_indices
  
```

Frame DAG

```

Clear → Count → Scan → Scatter → PackRanges
      → Forces → Integrate
      → Deposit → Convolve → Growth
      → TransportDenom → TransportGather
      → Sample → Render
  
```

Compile only the passes the active rules declare.

Rule	Uses
Mohr	hash + Forces + Integrate
Field Life	Convolve + MaCE pair
Lenia	Convolve + Growth
Particle Lenia	Deposit + Convolve + Sample + Integrate
Hybrid	union

Plugin surface (logical, not yet code)

```

Rule.passes() -> [Pass]
EulerianRule.convolve / update
LagrangianRule.forces / integrate
HybridRule.deposit / sample
  
```

Backends

Backend	Role
Python ece/	oracle, tests, small N

Backend	Role
wgpu / WGSL	interactive preview
CUDA on DGX Spark	capacity + future replay

Do not share one float shader between preview and bit-identical logs. Preview may use f32 exp. Replay, if required, uses LUTs and integer state on CUDA.

Bind-group partition (wgpu)

WebGPU cap is 8 storage buffers per stage. Group by pass family:

- Group 0 uniforms (SimParams)
- Group 1 particles
- Group 2 fields
- Group 3 matrix / kernel
- Group 4 hash tables

Rebind between passes. CUDA ignores this limit.

What is implemented now

Kernels + docs + configs + schemas. No host loop, no renderer, no scheduler. See docs/09-roadmap.md.

Determinism

BYTES_ARE_LAW applies to replay logs, not to the first interactive preview.

Why GPU floats are not enough

Vendors fuse FMA differently, approximate exp/sin differently, and schedule reductions in different orders. Two GPUs will diverge.

Preview vs replay

Path	Arithmetic	Promise
Python oracle	IEEE f64, fixed loops	tests match on one machine
wgpu preview	f32 WGSL	looks right, not a log
CUDA fixed-point (later)	Q16 position, Q8.24 velocity, exp LUT	cross-box replay

If / when a replay path is built

Persistent state becomes integer:

Quantity	Format
Position	Q16.16
Velocity	Q8.24
Matrix	Q1.15
Field mass	Q16.16

Replace exp, sin, cos with host-generated LUTs that never change. No data-dependent early exits except compile-time constants. Neighborhood walks are complete 3×3 / $3 \times 3 \times 3$.

State hash

xxHash3 (or BLAKE3) over:

```

frame u64 LE
particles N × (i32 x, i32 y, [i32 z], u32 type)
field    C × X × Y × [Z] × i32  (if present)
matrix   K² × i16

```

Derived buffers are functions of that state and are not hashed.

Tests that exist today

Python invariants in tests/. They lock math, not cross-vendor GPU bits. A reproducibility registry is postponed until a fixed-point backend exists.

DGX Spark notes

128 GB unified memory is a **capacity** win. GB10-class bandwidth is not an H100. Design for locality, mixed precision, and graphs. Do not plan interactive $1024^3 \times 64$ channels.

Honest first budget

Item	Size
$256^2 \times 8$ ch $\times 2$ buffers f32	~4 MB
$256^3 \times 8$ ch $\times 2$ buffers f32	~1 GB
4 M particles $\times 32$ B	~128 MB
cuFFT scratch (256^3)	hundreds of MB
7B Q4 VLM (later)	~5 GB

$512^3 \times 16$ is a research config, not the default.

What unified memory is for

- Host-side clustering / BUNCH / graph tools reading particle buffers without a PCIe staging stall.
 - Dumping replay frames to NVMe while the sim runs.
 - Co-locating an evaluator model later.
- It does not make 3D convolution cheap.

Stack split

Work	Tool
Field FFT $\geq$ radius ~6 in 3D	CUDA / cuFFT / CuPy
2D preview, UI	wgpu
Oracle / CI	Python ece/

Dawn-on-Spark is extra risk. Use CUDA on that box for the heavy field path.

3D memory warning

Grid	One f32 volume
256^3	64 MB
512^3	512 MB
1024^3	4 GB

Double-buffer $\times$ channels $\times$ scratch multiplies fast. Cap channels and resolution independently in config.

Bandwidth habits

- bf16 / fp16 for convolution payloads; keep MaCE mass in f32.
- Spatial hash with Morton order if the force kernel is cache-cold.

- CUDA graphs once the pass list is stable.
- Persistent kernels only after M1 is correct.

Roadmap

Cut from the original catalog. The catalog described a company-sized surface. This is the build order that can actually finish.

M0 — done

- Corrected Mohr tent (no extra `r_max`)
- Exponential friction
- Two-pass MaCE + torus wrap
- Snapshot hash tables
- Invariant tests (15 passing)
- This document tree, configs, schemas, WGSL fragments

M1 — 2D Mohr preview

- Host loop calling ece or a wgpu port of `shaders/mohr_force.wgsl`
- Live $K \times K$ matrix editor
- Torus wrap, bins, integrate
- Headless frame stepper for CI

Exit: a 6-species soup that matches the Python oracle on $N \leq 256$ for a few hundred steps within f32 tolerance.

M2 — Field Life 2D

- Convolution + `mace_denom` + `mace_gather`
- Mass-invariance test on GPU (readback sum per channel)
- Ring kernel from config

Exit: per-channel mass holds; creatures do not evaporate.

M3 — Lenia growth on the same field buffers

- Growth bell plugin
- Orbium config in `configs/lenia_orbium.toml`

Exit: Orbium persists or dies for documented reasons, not because of a broken clip.

M4 — PIC coupling

- Deposit particles $\rightarrow$ fields (sorted-bin add, not CAS-f32 on Spark)
- Sample fields $\rightarrow$ particles
- Particle Lenia as a plugin on that pair

M5 — CUDA path on Spark

- cuFFT for large kernels
- Optional fixed-point replay
- 256^3 default, 512^3 opt-in

Explicitly not scheduled

- Co-local 7B VLM steering (design only: `docs/12-ai-evaluator-deferred.md`)

- Full egui IDE / DAG node editor
- crates.io release matrix
- 64-species food webs
- 1024^3 volumes

Mapping onto Vita, Firebreak, Entelechy

These are product boundaries, not a merge plan.

Vita / Firebreak

Vita is a Conway-line sandbox (discrete Life + canopy / heightmap / JSON export / Firebreak mechanics). That is Life-like + optional continuous height, not Mohr.

Vita need	ECE piece
Mass that does not evaporate	MaCE / Flow-Lenia transport
Discrete Life	separate grid update, not the tent
Firebreak as a barrier field	extra Eulerian channel sampled by the CA
JSON exports	dump ρ and rule params, not particle SoA

Do not drive Vita's tick with Mohr forces. If a hybrid is wanted, Firebreak is a field that the CA reads, not a species in A.

Entelechy

Entelechy is the visual graph workbench (React Flow ports / handles / wires, JSON lane packets, object registry). ECE rules become graph nodes later; they are not nodes today.

Suggested port types when the graph wraps ECE:

Port	Buffer
particles	SoA pos/vel/type
field_c	one Eulerian channel
potential	convolution output
matrix	$K \times K$
kernel	radial samples
ranges	hash table

Edges: deposit, sample, potential, force. A lane packet can carry a SimConfig TOML blob plus a matrix checksum.

Daily Entelechy JSON packs stay in that repo. This repo stays the sim kernel + spec.

PlasmaForge / PIC

Deposit / sample is the same scatter-gather shape as charge deposit in a PIC plasma step. Share the pass idea, not the binary. Plasma needs Poisson / Maxwell; ALife does not.

SUBTERRA / game work

Game ticks can call M1 Mohr as a toy ecosystem layer. They must not take a dependency on MaCE 3D or the evaluator.

Config schema

Canonical JSON Schema: `schemas/sim.schema.json`. Human configs: `configs/*.toml`.

Required block

```
[simulation]
dt = 0.01
world = [1.0, 1.0]      # 2D torus
seed = 1

[species]
count = 6
matrix = "random"      # or "identity", "chase", "path"
range = [-1.0, 1.0]

[mohr]
particles = 4096
r_max = 0.08
beta = 0.3
gain = 1.0
lambda = 6.321          # -ln(0.9)*60 ≈ Mohr gamma 0.9 @ 60 Hz
```

Optional blocks

```
[field]
dims = [256, 256]
channels = 6
strength = 1.0
crowding_lambda = 0.1
transport_beta = 4.0
kernel = "ring"

[lenia]
mu = 0.15
sigma = 0.015
dt = 0.1

[hash]
cell = 0.08             # >= r_max
grid = [13, 13]

[io]
headless = true
frames = 1000
hash_every = 0
```

Unknown keys are errors. Do not silently ignore a misspelled `r_max`. The schema uses "additionalProperties": false on each object.

Matrix modes

Mode	Fill
random	U(range), seed from <code>simulation.seed</code>
identity	+1 diagonal, small negative off-diagonal

Mode	Fill
chase	cyclic +attract / -flee
path	load <code>matrix_path</code> as row-major f32 text

Validation rules

- `beta` $\in (0,1)$
- `r_max` > 0
- `lambda` ≥ 0
- `dt` > 0
- `species.count` == `matrix` side if a file is loaded
- `hash.cell` $\geq r_max$ unless `hash.neighborhood` is set to 5

Co-local AI evaluator — design only

Not scheduled. Written down so the idea does not get rebuilt from memory when M5 is actually done.

Why it waits

A VLM cannot debug a wrong tent, a leaking MaCE gather, or a scrambled hash table. Hand-edit the matrix until you can name a creature without a model.

Contract (when built)

Local multimodal model only. No network.

```
generate_multimodal(system, user, image, opts) -> text
last_embedding() -> optional [f32]
```

Stub backend returns score 0 and no deltas so the sim runs without a model.

Prompt shape

System: ALife critic. Score 0–1 against a target. Propose sparse (i, j, δ) with $\delta \in [-0.1, 0.1]$. JSON only.

User: target sentence, a few numeric stats (mean speed, spatial entropy, cluster count), 256² thumbnail.

Steering

$$v \leftarrow \beta v + (1-\beta) \delta$$

$$M \leftarrow \text{clip}(M + \eta v, -1, 1)$$

Default $\eta=0.5$, $\beta=0.7$. Saturation decay if a cell is pushed the same way for N evals. Reject updates that collapse $\text{std}(M)$ below 0.1.

Targets that work

Visible structure: “ring of chasers”, “stable bilayer”, “hollow shell with a core”.

Targets that do not work: periods and counts the still frame cannot show (“divides every 100 frames”).

Cadence

Default every 300 frames. Adaptive: double interval on plateau, halve on a jump. Evaluator thread separate from the GPU tick.

Memory on Spark (if enabled later)

7B Q4 ~5 GB plus KV cache. Leaves the 256³ sim untouched. Full frame readback is for offline analysis, not the live loop.

File index

Path	What it is
README.md	Entry point
CORRECTIONS.md	Deltas vs the original catalog
CHANGELOG.md	Dated notes
pyproject.toml	Package metadata
pytest.ini	Test path
justfile	Local commands
ece/__init__.py	Public surface
ece/mohr.py	Tent + $O(N^2)$ accelerations
ece/integrate.py	Friction + Euler + wrap
ece/mace.py	Affinity + two-pass MaCE
ece/bins.py	Snapshot hash
ece/config.py	TOML loader + validation
ece/matrix.py	Matrix generators
ece/step_mohr.py	CPU Mohr stepper (M1 host)
shaders/mohr_force.wgsl	Corrected force fragment
shaders/mace_2pass.wgsl	Denom + gather
tests/test_mohr.py	Force invariants
tests/test_mace.py	Mass / torus / softmax
tests/test_bins_integrate.py	Hash + friction
tests/test_config.py	Schema / TOML
tests/test_step_mohr.py	Stepper smoke
configs/*.toml	Named soups
schemas/sim.schema.json	Config JSON Schema
docs/00-15	Design set
docs/handbook.pdf	Printable compilation
tools/build_handbook.py	Handbook builder

Shader fragments

These are math-correct snippets, not a full wgpu project.

`shaders/mohr_force.wgsl`

- `mohr_force(r, a, beta)` — dimensionless tent
- `wrapped_delta_2d` — minimum image
- Force loop body multiplies by `params.gain` only
- Integrate uses `exp(- lambda * dt)`

`shaders/mace_2pass.wgsl`

- `mace_denom` writes `Z`
- `mace_gather` reads `Z` and wraps with `(x % n + n) % n`
- One channel shown; dispatch once per channel or flatten `CXY`

Bindings the host must supply

Mohr force pass: `SimParams`, positions, sorted indices, ranges, matrix, accel out.

MaCE: `SimParams` with `field_w/h` and `transport_beta`, affinity, `rho_in`, `denom`, `rho_out`.

Not provided

Prefix-sum shaders, FFT, deposit CAS, render. Those belong to M1–M4 hosts, not this reference.

Testing

```
python3 -m pytest tests/ -q
```

21 tests as of 0.1.0.

File	Locks
test_mohr.py	wall, tent peak, horizon, scale invariance, asymmetry, wrap
test_mace.py	per-channel mass, uniform fixed point, corner torus, softmax gather, crowding
test_bins_integrate.py	exclusive scan, frozen snapshot, Mohr-gamma conversion, wrap
test_config.py	TOML load, chase matrix, channel mismatch, unknown keys
test_step_mohr.py	torus stay-in, binned vs dense agreement

GPU tests do not exist yet. When they do, mass readback and a small-N oracle compare against `ece.mohr.mohr_accelerations` are the first two.

Corrections applied to the catalog / ECE spec

These replace the corresponding formulas in `particlelifesim.txt`. The Python package `ece/` is the executable source of truth.

1. Mohr force — drop the extra `r_max`

Was (catalog acceleration):

```
accel += F * r_max * unit(other - me)
```

Correct:

```
r = dist / r_max
F = tent(r, a, beta)          # dimensionless
accel += gain * F * unit(other - me)
```

`gain` defaults to 1. Peak mid-range force is then exactly the matrix entry `a`, independent of world scale. Scaling the world and `r_max` together must not change accelerations (see `test_force_independent_of_rmax_scale`).

Contact wall ($r < \beta$) stays species-independent and strictly repulsive. Clusters-style per-pair (r_1, r_2, f_1, f_2) may be added later; f_1 must not be attractive.

2. Friction — explicit time constant

Was: $v *= \gamma ** (60 * dt)$ (assumes 60 Hz)

Correct: $v = \exp(-\lambda dt)$

Conversion if you want Mohr's feel: $\lambda = -\ln(\gamma) * 60$.

3. MaCE — two passes, torus wrap, no border clip

Pass A — denominators

```
Z[x] = sum_{y in N(x)} exp(beta * A[y])
```

Pass B — gather

```
rho'[x] = sum_{y in N(x)} rho[y] * exp(beta * A[x]) / Z[y]
```

$N(x)$ is the 9-cell (2D) or 27-cell (3D) neighborhood **including self**, wrapped on the torus. Do not continue on out-of-bounds. Clipping leaks mass at the box face.

Invariant: $\text{sum}(\text{rho}'_c) == \text{sum}(\text{rho}_c)$ per channel, up to float rounding. Momentum is not conserved.

4. Spatial hash — freeze the snapshot

Keep four distinct arrays. Never scan a counter you later reuse as a write cursor.

Buffer	Role
<code>counts</code>	frozen particle-per-bin snapshot
<code>offsets</code>	exclusive prefix sum of <code>counts</code>

Buffer	Role
ranges	(offsets[b], offsets[b] + counts[b])
cursor	scatter write head, discarded after the pass

cell_size >= r_max if you walk a 3×3 neighborhood.

5. Scope cut (do not implement from the catalog yet)

M1 is 2D Mohr + matrix + these tests. Not in M1:

- 3D 512³ volumes
- 64-species Field Life
- CAS atomic f32 deposit
- co-local VLM
- full egui IDE / release matrix

wgpu is the interactive preview. Bit-identical replay, if required, lives on a fixed-point CUDA path later — not in WGSL floats.

6. DGX Spark budget for the first real run

Default: 256² or 256³ × 8 channels, 1–4 M particles, cuFFT when kernel radius > ~6 in 3D. 128 GB is capacity. GB10-class bandwidth is not an H100. Locality and mixed precision first.

Changelog

0.1.0 — 2026-09-28

- Implemented review corrections as executable kernels.
- Mohr tent: removed extra `r_max` scale; added `gain`.
- Friction: $\exp(-\lambda \, dt)$ with Mohr-gamma conversion helper.
- MaCE: two-pass denom + gather, torus wrap, per-channel mass tests.
- Spatial hash: frozen snapshot, exclusive scan, separate cursor.
- Document set docs/00–13, JSON schema, five TOML configs.
- CPU Mohr stepper with binned vs dense agreement test.
- WGSL fragments matching the corrected math.
- Printable handbook builder.